

AURA

Automated Urgent Relief Allocation

Diploma in Computer Science with Artificial Intelligence

Final Project – Proposal DCSAI-25.2-F



School of Computing and Engineering
National Institute of Business Management Sri
Lanka

NATIONAL INSTITUTE OF BUSINESS MANAGEMENT (NIBM)

Diploma in Computer Science with Artificial Intelligence

FINAL YEAR PROJECT PROPOSAL

Project Title: AURA

Group: 7

Submitted by:

Aakil Ahamad– KIC-DCSAI-252-F-037

Muzaffer– KIC-DCSAI-252-F-045

Azeezullah– KIC-DCSAI-252-F-032

Abdullah – KIC-DCSAI-252-F-014

Submitted to:

National Institute of Business Management

Date: 26th March 2026

Introduction

1.1 Introduction of the Website

AURA stands for Automated Urgent Relief Allocation. It is a new website we are building to help out during bad times. When a crisis hits, people need supplies fast. Our website connects big donation hubs directly to local village leaders. These leaders are called Grama Niladhari or GN officers. AURA makes sure food and medicine get to the right people without wasting time.

1.2 Website Structure

The website is built with three main parts working together. First, we use MySQL to keep all our data organized. It remembers users, locations, and all the inventory. Second, we use Python for the main web engine. Python makes the site run fast and handles the daily traffic. Third, we connect a special logic tool called Prolog. Prolog is like a smart brain. The Python side asks Prolog hard questions, and Prolog sends back the most logical answer.

1.3 Current Operations in Website

The website does a few main jobs to keep things moving. It sorts all the donated goods into three priority levels. Red is for critical things like medicine. Orange is for basic food rations. Yellow is for shelter. The website also runs a live public board. This board shows real-time stats about what specific items are missing in different areas.

1.4 Users and Responsibilities Of Website

Three main groups of people will use AURA.

Super Admins: These are big charity groups or NGOs. They update the master list of supplies. They do not have to read every request because the system sorts them out.

GN Officers: These are the people on the ground. They fill out forms to ask for standard supplies. They can also post public requests if their village needs something really unique.

Donors: Regular people can log in and look at the public board. If a donor has the exact thing a GN officer is asking for, they can book the request. Then they hand the supplies directly to the officer.

1.5 Problem Definition

Right now, handing out relief supplies is very messy and slow. Big charity centers get thousands of boxes. Workers have to read every single request form by hand. This takes way too long. Also, sometimes a village gets hit hard and needs a very specific item. The big warehouse might not have it. The normal system gets stuck here. People wait too long for help because the old way of doing things is just too disconnected.

1.6 Project Objectives

We have a few simple goals for AURA. We want to stop the long delays in handing out help. We want to build an automated system that pushes the worst emergencies to the top of the list right away. We want to let normal donors step in and help with unlisted items directly. We also want to make sure that if a vital medicine runs out, the system can instantly suggest a safe backup choice.

1.7 Proposed Solution

To fix these problems, we are putting AURA together. The Python system handles the speed and the user accounts. But the real fix is the Prolog logic engine. If a GN officer asks for infant supplies and says the roads are flooded, Prolog sees that. It instantly flags it as a top priority. Prolog also acts as a medical expert. If a drug is out of stock, it logically picks a substitute. Finally, we add the direct donor pipeline. This lets regular people hand special supplies straight to the local GN officer. It skips the slow warehouse completely.

Chapter 2

Methodology

2.1 Introduction

Let's look at how we will actually build AURA. This chapter is all about our game plan. We need a solid map to get from our ideas to a working website. Here is how we will do the work step by step.

2.2 Data Collection Methods

We need real facts to make this work. First, we will talk to actual GN officers. We want to know how they ask for help right now. Then we will look at old disaster records. This helps us see what supplies run out the fastest. We will also ask doctors about safe medicine swaps. We need that info to teach our Prolog logic engine.

2.3 Software Process Model

We are going to use the Agile method. This just means we build the project in small pieces. We do not want to wait months to see if something works. We will build a small part, test it, and show it to people. If it needs fixing, we fix it right away. Then we move on to the next piece. It keeps us moving fast.

2.4 Software Development Tools

Category	Tool / Technology	Purpose
Frontend Development	HTML, CSS	Runs the main website brain. It makes everything load up super quick.
Backend Development	Python, FastAPI	Acts as the logic tool. It figures out priorities and finds replacement medicines.
AI Component	Prolog	To build the intelligent travel suggestion system using logical rules.
Database Management	MySQL	Holds the database. It is really good at keeping big lists of records straight.
IDE / Code Editor	Visual Studio Code	To write, debug, and organize code efficiently.
Testing Tools	Manual Testing, Console Outputs in Python	To check functionality, identify bugs, and verify expected results.
Web Hosting Platform	XAMPP (Localhost) or Free Hosting Service	To host and demonstrate the website for presentation and testing.

2.5 Testing Strategies

We have to be sure the site does not break during a real emergency. We will test every small piece of code first. Then we will test how Python and Prolog talk to each other. We are actually using Prolog to make up really tricky test problems. This helps us see if the system can handle weird requests or heavy traffic without crashing.

2.6 Implementation Plan

Putting it all together will happen in stages. First, we will set up the MySQL database and get the user accounts working. Next, we will build the Python backend and make sure the basic forms work. After that, we will plug in the Prolog engine to handle the smart sorting and the medical logic. Last, we will open up the public board so donors can start helping directly.

Sprint 1: Setup and Database

- Set up the MySQL database to hold your data.
- Make the database store user accounts and the master supply list.
- Build the simple login screens so people can sign in safely.
- Start the frontend using HTML and CSS so it loads quick.

Outcome: The database is ready and users can log in.

Sprint 2: Main Website Forms

- Write the Python and FastAPI backend code.
- Create the blank forms for GN officers to ask for help.
- Build the viewing screen for the big charity groups.

Outcome: GN officers can fill out requests and admins can read them.

Sprint 3: Adding the Smart Logic

- Plug in the Prolog logic engine.
- Teach Prolog how to sort the worst emergencies to the top.
- Feed medical facts into Prolog so it can pick safe backup drugs.
- Make sure Python and Prolog pass messages to each other perfectly.

Outcome: The system sorts priority levels like red for medicine and picks drug swaps on its own.

Sprint 4: Public Board and Final Testing

- Finish the live public board for normal donors.
- Let donors book special requests to help directly.
- Test the system hard using Prolog to throw tricky problems at it.
- Fix any mistakes in the code right away.

Outcome: The site is fully tested and AURA is ready to go live.

Chapter 3

Timeline

The Gantt Chart of the timeline is given in Figure 3

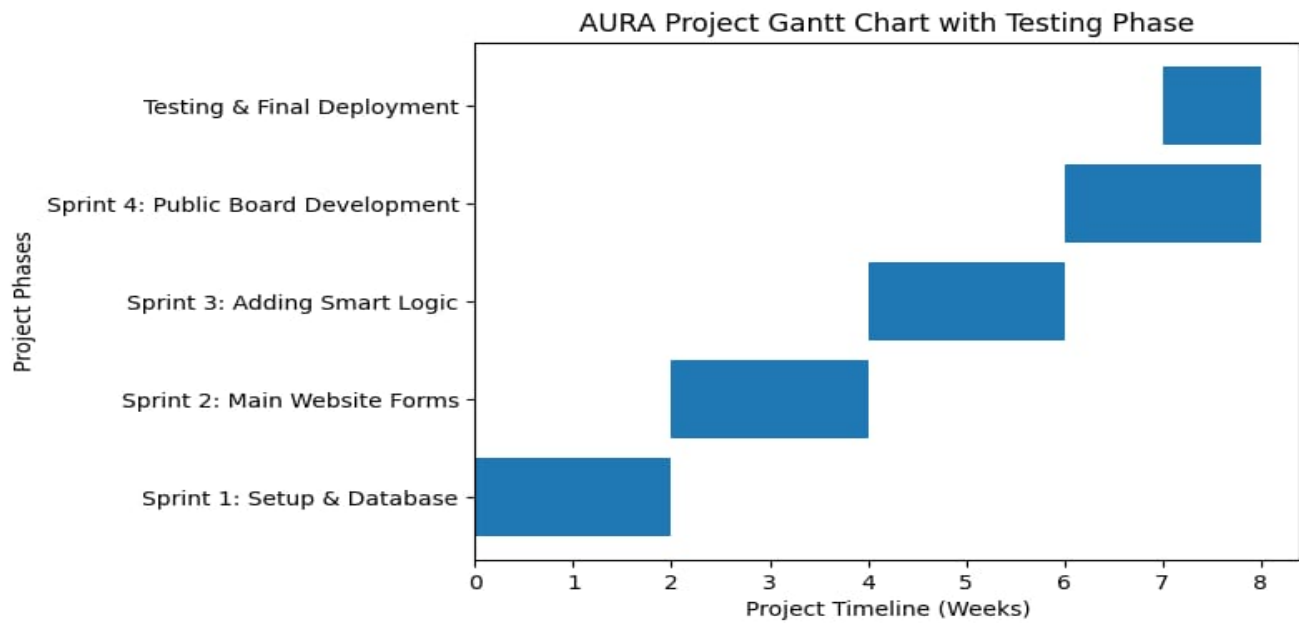


Figure 3.1 : Gantt Chart